

12. AGEX SDK

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX Platform의 공식 기능과 Resource Contract를 개발자가 각 Programming Language와 개발 환경에서 안전하고 일관되게 사용할 수 있도록 하는 **AGEX SDK의 구조, 책임, 기능, Packaging, Versioning, Authentication, Tenant Context, Resource Client, Runtime Client, Plugin SDK, Model Adapter SDK, Testing, CLI 연계 및 Acceptance** 기준을 정의한다.

AGEX SDK는 단순 HTTP API Wrapper가 아니다.

AGEX SDK는 개발자가 AGEX의 다음 핵심 개념을 잘못 구현하지 않도록 Platform Contract를 개발 언어 수준으로 제공한다.

- Tenant
- Resource
- Version / Revision
- Agent
- Workflow
- Execution
- Knowledge
- Memory
- Plugin
- Model Gateway
- Governance
- Event
- Audit
- Usage

AGEX SDK의 목적은 개발자가 직접 다음을 반복 구현하지 않도록 하는 것이다.

- Authentication 처리

- Tenant Context 전달
- Request Header
- Idempotency
- Pagination
- Error Mapping
- Optimistic Concurrency
- Long-running Execution Tracking
- Streaming
- Event/Webhook 처리
- Resource Serialization
- Schema Validation

본 문서는 AGEX API Specification과 API Guide를 기반으로 하며, API와 SDK 간의 계약 관계를 공식적으로 정의한다.

2. AGEX SDK의 공식 정의

AGEX SDK는 AGEX Platform Contract를 개발 언어별 Typed Client, Resource API, Runtime Interface, Extension Framework, CLI 및 Test Tool 형태로 제공하여 개발자가 AGEX의 공식 Resource와 실행 기능을 안전하고 일관된 방식으로 사용할 수 있도록 하는 공식 Developer Platform이다.

AGEX SDK는 AGEX API를 대체하지 않는다.

구조는 다음과 같다.

Developer

|



AGEX SDK

|



AGEX API



AGEX Platform

API가 Source of Truth이고 SDK는 그 Contract를 언어별로 표현하는 공식 Client Layer다.

3. SDK 설계 목표

AGEX SDK는 다음 목표를 갖는다.

1. AGEX API Contract를 언어별 Type으로 제공한다.
 2. Authentication을 표준화한다.
 3. Tenant Context 전달을 표준화한다.
 4. Resource CRUD와 Lifecycle 작업을 일관되게 제공한다.
 5. Long-running Execution을 쉽게 처리할 수 있게 한다.
 6. Streaming을 Provider와 무관한 방식으로 제공한다.
 7. Error를 Typed Exception/Error로 변환한다.
 8. Pagination을 표준화한다.
 9. Idempotency와 Concurrency를 쉽게 사용할 수 있게 한다.
 10. Plugin 개발을 위한 공식 Extension SDK를 제공한다.
 11. Model Provider Adapter 개발을 지원한다.
 12. Local/Test 환경을 지원한다.
 13. API와 SDK 간 Contract Drift를 방지한다.
 14. 특정 Model Provider 또는 Cloud에 종속되지 않는다.
 15. SDK가 Security Boundary를 우회하지 못하도록 한다.
-

4. SDK 설계 원칙

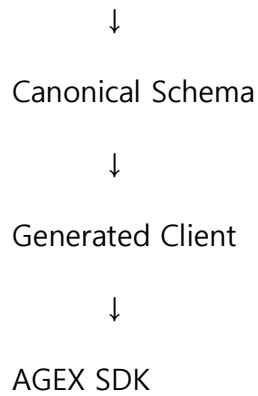
AGEX SDK는 다음 원칙을 따른다.

- API Contract First
- Schema First
- Generated Core Client
- Handwritten Developer Experience Layer
- Type Safe
- Async First
- Streaming Native
- Tenant Aware
- Version Aware
- Provider Agnostic
- Extension Friendly
- Secure by Default
- Retry Safe
- Idempotency Aware
- Backward Compatible
- Observable
- Testable
- Minimal Magic
- Explicit Security Context

5. SDK와 API의 관계

가장 중요한 원칙은 다음이다.

API Specification



SDK에서 먼저 기능을 정의하고 API를 나중에 맞추지 않는다.

SDK가 제공하는 모든 공식 Platform 기능은 AGEX API 또는 공식 Runtime/Extension Contract에 근거해야 한다.

6. Canonical Schema

AGEX의 Canonical Schema는 다음의 Source of Truth가 된다.

- Resource Schema
- API Request/Response
- Event Schema
- SDK Types
- Plugin Manifest
- 일부 Runtime Contract

예:

schemas/

├─ agent.schema.json

├─ workflow.schema.json

├─ execution.schema.json

├─ plugin.schema.json

└─ event.schema.json

SDK Type을 수동 정의한 뒤 서버 Schema와 별도로 유지하는 구조를 지양한다.

7. SDK 계층 구조

AGEX SDK는 다음 계층으로 구성하는 것을 권장한다.

AGEX SDK

|

├─ Core Client

├─ Authentication

├─ Tenant Context

├─ Resource Clients

├─ Runtime Client

├─ Streaming

├─ Event / Webhook

├─ Plugin SDK

├─ Model Adapter SDK

├─ Local/Test SDK

└─ Developer Utilities

8. Core Client

Core Client는 모든 SDK 기능의 기본 진입점이다.

개념 예:

```
const agex = new AGEXClient({
  endpoint: "https://api.example.com",
  credentialProvider,
  tenantId: "ten_123"
```

```
});
```

Python 예:

```
agex = AGEXClient(  
    endpoint="https://api.example.com",  
    credential_provider=credential_provider,  
    tenant_id="ten_123"  
)
```

9. Client Configuration

기본 Configuration에는 다음이 포함될 수 있다.

- Endpoint
- Credential Provider
- Tenant ID
- Timeout
- Retry Policy
- User Agent
- SDK Version
- Correlation Provider
- Transport Configuration

예:

endpoint

tenant

credential

timeout

retry

telemetry

10. Credential Provider

SDK는 Credential 값을 직접 받는 방식보다 Credential Provider 추상화를 지원해야 한다.

예:

Static Token Provider

OAuth Provider

Workload Identity Provider

API Credential Provider

Custom Credential Provider

11. Credential Provider Interface

개념:

```
interface CredentialProvider {  
    getAccessToken(): Promise<string>;  
}
```

또는 보다 구조화된 형태:

getCredential()

refresh()

invalidate()

SDK가 Token을 Source Code에 저장하게 유도하지 않는다.

12. Credential Refresh

OAuth 또는 Short-lived Credential을 사용하는 경우 SDK가 필요 시 Refresh를 지원할 수 있다.

단, Refresh 실패 시 무한 반복하지 않는다.

13. Tenant Context

Tenant Context는 SDK의 First-Class Concept다.

예:

```
const client = agex.withTenant("ten_123");
```

또는:

```
tenant = agex.tenant("ten_123")
```

모든 Tenant-owned API는 해당 Context를 사용한다.

14. Tenant Context 안전성

다음과 같은 Code를 장려하지 않는다.

```
client.getResource(resourceId)
```

후 Client 측에서 tenant 확인

Tenant Context는 요청 시점부터 서버 Contract에 포함되어야 한다.

15. Multiple Tenant

사용자가 여러 Tenant를 사용하는 Application이라면 Client 전체를 재생성하기보다 안전한 Scoped Client를 제공할 수 있다.

예:

```
const tenantA = agex.withTenant("ten_a");
```

```
const tenantB = agex.withTenant("ten_b");
```

Scoped Client는 내부적으로 Context를 분리해야 한다.

16. Resource Client 구조

대표적으로 다음 Domain Client를 제공한다.

agex.tenants

agex.agents

agex.workflows

agex.executions

agex.knowledge

agex.memory

agex.plugins

agex.models

agex.approvals

agex.governance

agex.events

agex.artifacts

agex.usage

agex.audit

17. Agent Client

개념:

agex.agents.create()

agex.agents.get()

agex.agents.list()

agex.agents.update()

agex.agents.publish()

agex.agents.activate()

agex.agents.suspend()

```
agex.agents.execute()
```

18. Agent 생성 예

```
const agent = await agex.agents.create({
  name: "example-agent",
  specification: {
    goal: "Handle the requested task.",
    autonomyLevel: "L1",
    modelPolicy: {
      profileId: "mdlprof_default"
    }
  }
});
```

실제 Field 이름은 Canonical Schema를 따른다.

19. Agent Publish

```
const published = await agex.agents.publish(agent.id);
```

Publish 결과에는 새 Version이 포함될 수 있다.

20. Agent 실행

```
const execution = await agex.agents.execute("agt_123", {
  version: 4,
  input: {
    message: "..."
  }
});
```

```
});
```

SDK는 Execution ID를 반환해야 한다.

21. Execution Handle

개발 편의를 위해 SDK는 Execution Handle을 제공할 수 있다.

예:

```
const execution = await agex.agents.execute(...);
```

```
const result = await execution.wait();
```

그러나 내부적으로는 API Polling/Event/Streaming을 사용해야 한다.

wait()가 단일 HTTP Connection을 무기한 유지한다는 의미는 아니다.

22. Execution Handle 기능

지원 가능 기능:

get()

wait()

cancel()

stream()

onEvent()

getResult()

예:

```
await execution.cancel();
```

23. Execution 상태

SDK는 Raw String보다 Typed Enum을 제공할 수 있다.

예:

```
type ExecutionState =  
  | "queued"  
  | "running"  
  | "waiting"  
  | "waiting_for_approval"  
  | "retrying"  
  | "completed"  
  | "failed"  
  | "cancelled"  
  | "timed_out";
```

Unknown Future State 처리를 고려한다.

24. Workflow Client

제공 기능:

create

get

list

update

publish

activate

execute

예:

```
const execution = await agex.workflows.execute("wf_123", {  
  version: 2,
```

```
input: {...}  
});
```

25. Workflow Execution Handle

Agent와 동일한 Execution Contract를 사용할 수 있다.

즉:

Agent Execution

Workflow Execution

을 별도의 완전히 다른 Client Model로 만들지 않는다.

26. Knowledge Client

지원 기능:

collections.create

collections.get

sources.create

sources.sync

retrievals.create

또는 언어별 개발 편의를 위해 Namespace 구조를 제공한다.

27. Knowledge Retrieval 예

```
const results = await agex.knowledge.retrieve({  
  collectionIds: ["knc_1"],  
  query: "runtime recovery",  
  limit: 5  
});
```

Result는 Citation Metadata를 포함해야 한다.

28. Memory Client

기능:

create

propose

retrieve

delete

list

Memory Scope를 Typed Value로 제공한다.

예:

scope: "session" | "agent" | "user" | "tenant"

29. Plugin Client

Plugin 관련 기능:

plugins.list

plugins.get

installations.create

installations.enable

installations.disable

actions.execute

30. Plugin Installation과 Definition 구분

SDK에서도 다음을 구분해야 한다.

Plugin Definition

Plugin Package

Plugin Installation

Plugin Action

하나의 plugin Object에 모든 개념을 혼합하지 않는다.

31. Model Client

지원:

models.list

models.get

modelProfiles.create

modelProfiles.update

modelExecutions.create

Provider-specific API를 SDK의 Public Surface에 직접 노출하지 않는 것이 원칙이다.

32. Model 실행

단순 Model Invocation이 필요할 경우:

```
const result = await agex.models.generate({  
  profileId: "mdlprof_123",  
  input: {...}  
});
```

내부적으로 Model Gateway를 사용한다.

33. Provider-neutral Result

Model Result는 Provider별 Raw Response가 아니라 공통 Result를 우선 제공한다.

예:

content

structuredOutput

toolCalls

usage

finishReason

providerMetadata

필요 시 제한적인 Raw Provider Metadata 접근을 별도로 제공할 수 있다.

34. Approval Client

지원 기능:

list

get

approve

reject

예:

```
await agex.approvals.approve("apr_123", {  
  comment: "Approved"  
});
```

35. Approval Safety

SDK가 승인 API를 호출한다고 Runtime Validation이 생략되어서는 안 된다.

승인 자체는 의사결정 Record일 뿐이다.

36. Governance Client

제공 가능:

- Risk
- Exceptions
- Reviews
- Evaluations
- Policy Simulation

예:

```
const result = await agex.governance.simulatePolicy({...});
```

37. Audit Client

Audit는 Read-Oriented Client를 기본으로 한다.

```
const events = await agex.audit.list({
  resourceId: "agt_123"
});
```

일반 SDK에 Audit Edit/Delete 기능을 제공하지 않는다.

38. Usage Client

지원:

- Tenant Usage
- Resource Usage
- Model Usage
- Runtime Usage
- Cost Summary

예:

```
const usage = await agex.usage.get({
  from,
```

```
to,  
  groupBy: "agent"  
});
```

39. Artifact Client

큰 파일과 Output을 처리한다.

기능:

createUpload

completeUpload

get

download

delete

40. Upload Strategy

대용량 File은 SDK가 Pre-signed Upload Flow를 추상화할 수 있다.

개념:

```
await agex.artifacts.upload(file);
```

내부적으로:

Create Upload

→ Upload

→ Complete

→ Validate

를 수행한다.

41. Pagination

SDK는 Cursor Pagination을 편리하게 처리해야 한다.

기본 방식:

```
const page = await agex.agents.list({ limit: 50 });
```

42. Async Iterator

지원 언어에서는 Async Iterator를 제공할 수 있다.

예:

```
for await (const agent of agex.agents.iterate()) {  
    ...  
}
```

내부적으로 Cursor Pagination을 사용한다.

43. Pagination 안전성

SDK가 사용자가 모르는 사이 모든 Resource를 Memory에 로드하는 API를 기본으로 제공하지 않는다.

예:

```
listAll()
```

은 대량 Dataset에서 위험할 수 있다.

44. Idempotency 지원

SDK는 Idempotency Key를 쉽게 사용할 수 있게 한다.

예:

```
await agex.agents.execute(id, input, {  
    idempotencyKey: key  
});
```

45. Automatic Idempotency

SDK가 자동 Key 생성을 지원할 수 있다.

하지만 네트워크 재시도에서 같은 논리 Operation은 같은 Key를 사용해야 하므로 Scope를 명확히 해야 한다.

46. Retry

SDK는 일부 Retry를 자동 지원할 수 있다.

자동 Retry 후보:

- Network Error
- 429
- 일부 5xx

자동 Retry 금지:

- Permission Denied
 - Policy Denied
 - Validation Error
 - Conflict
-

47. Retry Policy

설정 예:

```
retry: {  
  maxAttempts: 3,  
  backoff: "exponential",  
  jitter: true  
}
```

48. Retry-After

Server가 Retry-After를 제공하면 SDK는 이를 우선 고려해야 한다.

49. Optimistic Concurrency

SDK는 ETag 또는 Revision을 Developer-friendly하게 지원한다.

예:

```
await agex.agents.update(agent.id, patch, {  
    ifMatch: agent.etag  
});
```

50. Conflict Handling

Conflict 발생 시 SDK는 자동으로 최신 값과 Merge하지 않는다.

이는 Domain 의미를 훼손할 수 있다.

Typed Conflict Error를 반환한다.

51. Error Model

AGEX SDK는 HTTP Error를 공식 SDK Error로 Mapping한다.

예:

AuthenticationError

PermissionDeniedError

PolicyDeniedError

ValidationError

ConflictError

RateLimitError

QuotaExceededError

ProviderUnavailableError

ExecutionTimeoutError

AGEXInternalError

52. Stable Error Code

SDK Error에는 원본 AGEX Error Code를 유지한다.

예:

```
try {  
    ...  
} catch (err) {  
    if (err.code === "POLICY_DENIED") {  
        ...  
    }  
}
```

Message String Parsing에 의존하지 않는다.

53. Request ID

모든 SDK Error에 가능하면 다음을 포함한다.

- requestId
- correlationId
- statusCode
- AGEX error code

Support와 Debugging에 중요하다.

54. Error Detail 보호

SDK가 Internal Stack Trace나 Sensitive Server Detail을 Client Error에 추가하지 않는다.

55. Streaming

Streaming은 AGEX SDK의 핵심 기능이다.

지원 형태:

- Async Iterator
- Callback/Event Emitter
- Reactive Stream

언어 특성에 맞게 제공한다.

56. Streaming 예

TypeScript:

```
for await (const event of execution.stream()) {  
    console.log(event);  
}
```

Python:

```
async for event in execution.stream():
```

```
    ...
```

57. Streaming Event

대표:

- execution.started
- output.delta
- action.proposed

- approval.required
- execution.completed
- execution.failed

Raw Chain-of-Thought Event는 제공하지 않는다.

58. Streaming 중단

Network가 끊기면 SDK는 다음을 할 수 있어야 한다.

- Reconnect where supported
- Resume from cursor/event id
- Fallback to execution.get()

Execution 자체를 자동 취소하지 않는다.

59. Streaming과 State

반드시:

Stream State

≠

Execution State

를 유지한다.

60. Event Client

AGEX Event를 수신하거나 구독하는 Client를 제공할 수 있다.

기능:

subscriptions.create

subscriptions.delete

events.list

61. Webhook SDK

Webhook Receiver용 Helper를 제공할 수 있다.

예:

```
agex.webhooks.verifySignature(...)
```

62. Webhook Handler

개념:

```
const event = agex.webhooks.verifyAndParse({  
  headers,  
  body  
});
```

SDK가 Signature Verification을 쉽게 제공해야 한다.

63. Replay Protection

Webhook Helper는 Timestamp와 Event ID를 확인할 수 있게 해야 한다.

단, 최종 Dedup 저장은 Application 책임일 수 있다.

64. Observability

SDK는 다음 Context를 자동 전달할 수 있다.

- X-Request-ID
- X-Correlation-ID
- Trace Context
- SDK Version
- Runtime/Language Information

65. OpenTelemetry 연계

지원 언어에서는 OpenTelemetry Adapter를 제공할 수 있다.

목적:

Application Trace

→ AGEX API

→ Runtime Trace

연결.

66. SDK Telemetry

SDK 자체 Telemetry는 사용자 데이터나 Prompt 전체를 자동 수집하지 않는다.

가능한 Metadata:

- request latency
 - retries
 - errors
 - SDK version
-

67. Logging

SDK는 자체 Debug Logging을 지원할 수 있다.

하지만 다음은 Mask해야 한다.

- Authorization Header
 - Secret
 - Token
 - Raw Credential
-

68. HTTP Transport Abstraction

SDK Core는 HTTP Transport를 추상화할 수 있다.

장점:

- 테스트
 - Browser/Node 차이
 - Custom Enterprise Transport
 - Proxy/mTLS
-

69. Transport Interface

개념:

send(request)

stream(request)

SDK Business Logic이 특정 HTTP Library에 과도하게 결합되지 않도록 한다.

70. Timeout

SDK는 최소 다음 Timeout을 구분할 수 있다.

- Connection Timeout
- Request Timeout
- Streaming Idle Timeout

Execution 자체의 Timeout은 Platform Resource 설정이다.

71. Proxy

Enterprise 환경에서는 HTTP Proxy를 지원할 수 있다.

Proxy Configuration이 Tenant Security 정책을 우회해서는 안 된다.

72. mTLS

Private Enterprise 환경에서는 mTLS Client Configuration을 지원할 수 있다.

Certificate/Key는 SDK Source Code에 직접 포함하지 않는다.

73. Language Support 정책

초기 Tier 1 SDK는 다음을 권장한다.

- TypeScript / JavaScript
- Python

이 두 언어를 공식 First-Class SDK로 운영한다.

74. Tier 2 SDK

제품 성장 후 다음을 검토할 수 있다.

- Java
- Go
- .NET/C#

Enterprise 수요와 Ecosystem을 기준으로 결정한다.

75. Tier 3 / Community SDK

기타 언어는 Community SDK 또는 Generated Client로 제공할 수 있다.

Official Support Level을 명확히 표시한다.

76. SDK Support Level

예:

Tier 1 — Official

- Full Feature

- Release Support
- Integration Test
- Documentation

Tier 2 — Supported

- Major API Coverage
- Regular Release

Community

- Best Effort
- Community Maintained 가능

77. SDK Package Naming

일관된 Package Naming을 사용한다.

예:

@agex/sdk

agex-sdk

com.agex.sdk

실제 Brand Namespace는 Release 전 확정한다.

78. SDK Module 구조

TypeScript 예:

@agex/sdk

@agex/plugin-sdk

@agex/model-adapter-sdk

@agex/testing

Python:

agex

agex_plugin

agex_model_adapter

agex_testing

79. Core SDK와 Extension SDK 분리

Client SDK와 Plugin SDK를 하나의 큰 Package로 강제 결합하지 않는다.

Client SDK

Plugin SDK

Model Adapter SDK

Testing SDK

를 논리적으로 분리한다.

80. Plugin SDK 목적

Plugin SDK는 외부 개발자가 AGEX 호환 Plugin을 개발하도록 한다.

지원 기능:

- Manifest Types
- Capability Definition
- Action Handler
- Context
- Credential Reference
- Logging
- Error Mapping
- Test Harness
- Packaging

81. Plugin Handler 예

개념:

```
defineAction({  
  name: "items.get",  
  inputSchema,  
  outputSchema,  
  sideEffect: "read_only",  
  
  async handler(ctx, input) {  
    ...  
  }  
});
```

82. Plugin Execution Context

Plugin Handler에 제공할 수 있는 Context:

- tenant reference
- execution id
- action id
- credential resolver
- logger
- abort signal
- trace context

83. Plugin Context에서 금지

Plugin Handler가 다음에 직접 접근할 수 있게 하지 않는다.

- Core Database
- 다른 Tenant Secret
- Platform Admin Token

84. Plugin Credential Resolver

개념:

```
const credential = await ctx.credentials.resolve("external-api");
```

하지만 SDK가 반환하는 Credential의 사용 범위와 Lifetime을 최소화해야 한다.

더 강한 Sandbox에서는 SDK 자체가 HTTP Request에 Credential을 주입하고 Plugin Code에 Secret을 노출하지 않을 수도 있다.

85. Plugin Error Mapping

Plugin SDK는 다음과 같은 Error Helper를 제공할 수 있다.

- invalidInput
- authFailed
- rateLimited
- externalUnavailable
- timeout
- permanentFailure

Runtime이 Retry 여부를 판단하는 데 활용한다.

86. Plugin Test Harness

지원:

- Mock Tenant

- Mock Credential
- Network Mock
- Timeout
- Permission
- Schema Validation

87. Plugin Packaging Tool

SDK/CLI 연계를 통해:

validate

build

sign

package

publish

과정을 자동화할 수 있다.

88. Model Adapter SDK 목적

새 AI Model Provider를 AGEX Model Gateway에 연결하기 위한 공식 SDK다.

89. Provider Adapter Interface

공통 기능:

listModels

capabilities

generate

stream

embed

rerank

health

usage

Provider가 지원하지 않는 기능은 명확히 Unsupported로 처리한다.

90. Generate Input

공통 요청 구조:

- messages/context
 - output schema
 - tool definitions
 - model parameters
 - budget/limits
 - metadata
-

91. Generate Output

공통 결과:

- content
 - structured output
 - tool calls
 - finish reason
 - usage
 - provider metadata
-

92. Adapter Tool Calling

Adapter는 Provider Tool Calling을 AGEX 공통 Action Candidate로 변환한다.

Adapter가 실제 Tool을 실행하지 않는다.

93. Adapter Structured Output

Provider가 Structured Output을 지원하지 않는 경우 SDK Helper로 Parse/Validate를 제공할 수 있다.

Schema Validation 실패는 성공 응답으로 취급하지 않는다.

94. Adapter Streaming

Provider별 Streaming Event를 AGEX Common Event로 Normalize한다.

95. Adapter Error

Provider-specific Error를 AGEX 공통 Error로 Mapping한다.

96. Adapter Usage

Token 또는 Provider Usage를 AGEX 공통 Usage Record로 변환한다.

97. Adapter Test Harness

검증:

- Generate
- Streaming
- Structured Output
- Tool Calling
- Timeout
- Rate Limit
- Error Mapping

- Usage

98. Test SDK

AGEX Testing SDK는 Agent/Workflow/Plugin/Application Test를 지원한다.

99. Test Client

Local 또는 Integration Environment에서 API를 쉽게 호출할 수 있는 Fixture Client를 제공할 수 있다.

100. Mock Model

Testing SDK는 Deterministic Mock Model을 제공할 수 있다.

예:

When input matches A

→ return structured output B

101. Mock Tool

Agent Tool 호출을 예측 가능하게 테스트한다.

102. Mock Knowledge

Fixture 기반 Retrieval 결과를 반환할 수 있다.

103. Mock Memory

Memory Recall/Write Proposal을 테스트한다.

104. Execution Test Helper

예:

```
await expectExecution(execution)
    .toComplete()
    .toCallTool("items.get")
    .not.toCallTool("items.delete");
```

실제 API 설계는 언어에 맞게 구현한다.

105. Policy Test Helper

예:

```
expect action X to be denied
expect action Y to require approval
```

106. Tenant Isolation Test Helper

Test SDK는 Cross-Tenant Test를 쉽게 만들 수 있어야 한다.

예:

```
await expect(
    tenantA.agents.get(tenantBResource)
).rejects.toBePermissionDenied();
```

107. Contract Test

SDK Release마다 실제 API와 Contract Test를 수행한다.

SDK

↔ API Schema

↔ Server

108. Generated Client

Low-level API Client는 Schema/OpenAPI 기반으로 생성할 수 있다.

장점:

- Field Drift 방지
 - Type consistency
 - Endpoint coverage
-

109. Handwritten Wrapper

Generated Client 위에 개발자 친화적인 Layer를 제공한다.

예:

Generated:

```
createAgent(request)
```

High-level:

```
agents.create({...})
```

110. Generated Code 직접 수정 금지

생성된 Code를 직접 수정하지 않는다.

필요한 기능은:

- Generator
- Template
- Wrapper

에서 변경한다.

111. SDK Versioning

SDK Version은 API Version과 독립적이다.

예:

API v1

SDK 2.3.0

SDK가 지원하는 API Version 범위를 명시한다.

112. Semantic Versioning

SDK는 SemVer를 적용하는 것을 권장한다.

MAJOR.MINOR.PATCH

Major

Breaking SDK API 변경.

Minor

Backward-compatible 기능 추가.

Patch

Bug/Security Fix.

113. API Breaking Change와 SDK

API 새 Major Version이 나오더라도 SDK가 즉시 Major Version을 올려야 한다는 의미는 아니다.

SDK의 Public Interface Compatibility를 별도로 판단한다.

114. Deprecation

SDK 기능 폐기 절차:

Active

→ Deprecated

→ Migration Warning

→ Removal in Major Release

Security Emergency는 예외다.

115. Runtime Warning

Deprecated SDK Method 사용 시 Development 환경에서 Warning을 제공할 수 있다.

Production Log 과다 발생은 피한다.

116. Compatibility Matrix

공식 문서:

SDK API

SDK 1.x API v1

SDK 2.x API v1/v2 등

실제 지원 범위를 Release마다 관리한다.

117. Language Runtime Support

각 SDK는 지원 Runtime Version을 명확히 한다.

예:

- Node.js LTS
- Python supported releases
- Java LTS

EOL Runtime을 무기한 지원하지 않는다.

118. Browser SDK

TypeScript SDK는 Browser와 Server 사용을 구분할 수 있다.

Browser에서는 다음 기능을 제한한다.

- Service Credentials
 - Secret Management
 - High-Privilege Admin API
 - Plugin Development Runtime
-

119. Browser Authentication

Browser는 User-scoped Short-lived Token을 사용한다.

Long-lived API Secret을 Browser에 저장하지 않는다.

120. Server SDK

Backend/Server 환경에서는 다음을 지원한다.

- Workload Identity
 - Service Credential
 - mTLS
 - Long-running Integration
-

121. SDK Security

SDK 자체가 다음을 자동으로 Log하지 않는다.

- Token
 - Secret
 - Credential
 - Full Prompt
 - Full Memory
-

122. Credential Memory

가능하면 Token/Secret을 필요 이상으로 오래 Process Memory에 유지하지 않는다.

123. TLS

SDK는 HTTPS를 기본으로 강제한다.

Development Localhost 등 명시적인 개발 환경을 제외하고 Plain HTTP를 허용하지 않는 것이 좋다.

124. Certificate Validation

Production에서 TLS Certificate Validation을 끄는 옵션을 쉽게 제공하지 않는다.

필요한 Enterprise Private CA는 Trust Store 방식으로 지원한다.

125. SDK Retry Security

Authentication 실패를 자동 Credential 반복 호출로 폭증시키지 않는다.

401/403은 기본 자동 Retry 대상이 아니다.

126. SDK Input Validation

SDK는 개발 편의를 위해 Client-side Validation을 제공할 수 있다.

그러나 Server Validation이 최종 기준이다.

127. SDK Output Validation

서버 Response가 계약과 다르면 SDK는 Contract Error를 발생시킬 수 있다.

이는 Server/Proxy 이상 탐지에 유용하다.

128. Serialization

Date, Decimal, Enum, Binary 등 언어별 Representation을 일관되게 정의한다.

129. Timestamp

SDK는 Timestamp를 언어별 Date/Datetime Object로 변환할 수 있지만 원본 UTC 의미를 유지한다.

130. Decimal

Billing/Cost 값은 Float 손실 없이 처리해야 한다.

언어별 Decimal 또는 String 기반 Representation을 사용한다.

131. Enum Forward Compatibility

새 Server Enum이 기존 SDK에 나타날 수 있다.

SDK가 전체 Parsing Failure를 일으키지 않도록 Unknown Enum Strategy를 고려한다.

132. Resource Reference

SDK는 Resource 전체 Object를 요구하기보다 Resource Reference를 지원한다.

예:

```
agentId: "agt_123"
```

또는:

```
agent: {  
  id: "agt_123",  
  version: 4  
}
```

133. Version Pinning Helper

개발자가 Production Resource Version을 명시적으로 고정하기 쉽게 해야 한다.

예:

```
AgentRef.version("agt_123", 4)
```

134. Active Version Helper

개발 편의를 위해 Active Version 참조를 지원할 수 있지만 Production에서는 재현성 위험을 문서화한다.

135. Async First

네트워크 호출은 기본적으로 비동기 API를 제공한다.

Python에서도 Async SDK를 First-Class로 고려한다.

동기 Wrapper는 추가로 제공할 수 있다.

136. Cancellation Token

지원 언어에서는 Request Cancellation/AbortSignal을 제공한다.

예:

```
await client.agents.execute(..., { signal });
```

이는 Client Request Cancel과 Server Execution Cancel이 다름을 문서화해야 한다.

137. Server Execution Cancel

실제 Agent 실행을 취소하려면:

```
await execution.cancel();
```

을 사용한다.

138. Local SDK

Local Runtime용 Client를 제공할 수 있다.

예:

```
const agex = AGEXClient.local();
```

Local과 Cloud에서 최대한 동일 Client Interface를 사용한다.

139. Local Runtime 차이

Local 환경에서는 다음이 제한될 수 있다.

- Enterprise SSO
- Dedicated Runtime
- Full Marketplace
- Full Governance

그러나 Resource/Runtime 기본 Contract는 동일해야 한다.

140. CLI와 SDK 관계

AGEX CLI는 SDK를 사용할 수 있다.

CLI

→ SDK

→ API

SDK가 CLI에 종속되지는 않는다.

141. CLI Generated Types

CLI Input Validation에 동일 Canonical Schema를 사용할 수 있다.

142. SDK Documentation

각 SDK는 최소 다음 문서를 제공한다.

- Installation
- Authentication
- Quick Start
- Tenant
- Agent
- Workflow
- Execution
- Streaming
- Errors
- Pagination
- Advanced Configuration

143. Code Example 원칙

모든 공식 예제는 다음을 준수해야 한다.

- Secret 하드코딩 금지
- Tenant 명시
- Error 처리
- Execution 상태 처리
- 최신 공식 Contract 사용

144. Quick Start — TypeScript

개념:

```
import { AGEXClient } from "@agex/sdk";
```

```
const agex = new AGEXClient({
```

```

    endpoint: process.env.AGEX_ENDPOINT!,
    credentialProvider,
    tenantId: process.env.AGEX_TENANT_ID!
  });

const execution = await agex.agents.execute(
  "agt_123",
  {
    version: 1,
    input: {
      message: "Hello"
    }
  }
);

```

```
const result = await execution.wait();
```

```
console.log(result.output);
```

145. Quick Start — Python

개념:

```
from agex import AGEXClient
```

```
agex = AGEXClient(
    endpoint=endpoint,
```



```
credential_provider=credential_provider,  
tenant_id=tenant_id,  
)
```

```
execution = await agex.agents.execute(  
    "agt_123",  
    version=1,  
    input={"message": "Hello"},  
)
```

```
result = await execution.wait()
```

```
print(result.output)
```

146. Streaming 예

```
const execution = await agex.agents.execute(...);
```

```
for await (const event of execution.stream()) {  
    if (event.type === "output.delta") {  
        process.stdout.write(event.delta);  
    }  
}
```

Stream 종료 후:

```
const finalState = await execution.get();
```

으로 상태를 확인할 수 있어야 한다.

147. Error 처리 예

```
try {  
    await agex.agents.execute(...);  
} catch (error) {  
    if (error.code === "POLICY_DENIED") {  
        // 정책 거부 처리  
    } else if (error.code === "RATE_LIMITED") {  
        // 제한적 재시도  
    } else {  
        throw error;  
    }  
}
```

148. Pagination 예

```
for await (const workflow of agex.workflows.iterate({  
    state: "active"  
})) {  
    console.log(workflow.id);  
}
```

149. Idempotency 예

```
const execution = await agex.agents.execute(  
    "agt_123",  
    request,
```

```
{
    idempotencyKey: businessOperationId
}
);
```

네트워크 재시도 시 같은 Key를 유지한다.

150. Concurrency 예

```
const agent = await agex.agents.get("agt_123");
```

```
await agex.agents.update(
    agent.id,
    {
        name: "new-name"
    },
    {
        ifMatch: agent.etag
    }
);
```

151. SDK Configuration Precedence

설정 우선순위를 명확히 한다.

예:

Method Option

>

Scoped Client

>

Client Configuration

>

Environment Default

Secret/Endpoint가 의도치 않게 다른 Environment로 전환되지 않도록 한다.

152. Environment Profile

지원 가능:

development

staging

production

단, Production Credential과 Development Endpoint의 잘못된 조합을 탐지할 수 있으면 좋다.

153. SDK Middleware

고급 사용자를 위해 Middleware/Interceptor를 제공할 수 있다.

사용 예:

- Logging
- Metrics
- Custom Headers
- Trace

154. Middleware 금지 범위

Middleware가 다음을 우회할 수 있어서는 안 된다.

- Tenant Enforcement

- Authentication
 - Server Permission
 - TLS Validation
-

155. Custom Header

일부 Metadata Header 추가를 허용할 수 있다.

하지만 Authorization/Tenant Header를 임의 덮어쓰는 방식은 제한한다.

156. Request Hook

SDK가 Request Hook을 제공한다면 Secret/PII 노출 위험을 문서화해야 한다.

157. Response Hook

Monitoring 용도로 사용할 수 있지만 Full Content를 자동 저장하지 않는다.

158. SDK Performance

SDK 자체 Overhead는 최소화한다.

주요 최적화 대상:

- Serialization
 - Connection Pool
 - Streaming
 - Retry
 - Pagination
-

159. Connection Pooling

Server SDK는 HTTP Keep-Alive/Connection Pool을 지원한다.

160. Thread Safety / Concurrency Safety

SDK Client는 가능한 경우 Concurrent-safe하게 설계한다.

특히 Tenant Scoped Context를 Global Mutable State로 변경하지 않는다.

나쁜 예:

```
client.tenant = A
```

```
async call
```

```
client.tenant = B
```

동시 요청에서 Cross-Tenant 요청을 일으킬 수 있다.

161. Immutable Scoped Client

권장:

```
const tenantA = client.withTenant("A");
```

```
const tenantB = client.withTenant("B");
```

원본 Client 상태를 변경하지 않는다.

162. Multi-thread Python/Java

언어별 Thread/Async Safety 수준을 문서화한다.

163. SDK Resource Cache

SDK가 Resource Cache를 사용할 경우 Permission/Tenant/Version을 Cache Key에 포함한다.

Global Resource Cache에서 Tenant Data를 섞지 않는다.

164. Cache 기본 정책

SDK는 기본적으로 Platform Resource의 강한 Cache를 사용하지 않는 것이 안전하다.
필요 시 사용자 명시적 Cache Strategy를 지원한다.

165. Offline SDK

AGEX SDK는 일반적인 Offline-first SDK가 아니다.

Resource Source of Truth는 AGEX Platform이다.

Local Development Mock은 별도 기능으로 취급한다.

166. SDK Extension Points

허용 가능한 확장:

- Credential Provider
 - Transport
 - Telemetry Adapter
 - Retry Policy
 - Serializer 일부
 - Plugin Handler
 - Model Adapter
-

167. Extension Point Stability

공식 Extension Interface는 Public SDK Contract로 Version 관리한다.

Internal Class 상속을 Extension 방법으로 문서화하지 않는다.

168. SDK Code Generation Pipeline

권장:

Canonical Schema

- API Contract
 - Generator
 - Generated SDK Core
 - Compile/Test
 - High-level Wrapper
 - Package
-

169. SDK CI

최소 Pipeline:

Generate

- Format
 - Compile
 - Unit Test
 - Contract Test
 - Integration Test
 - Security Scan
 - Package
-

170. Golden Contract Test

대표 API Response Fixture와 Schema를 비교해 SDK Serialization Drift를 탐지한다.

171. Cross-Language Consistency

TypeScript와 Python SDK는 이름이 완전히 동일할 필요는 없지만 개념적 동작은 일치해야 한다.

예:

agents.execute

executions.get

workflows.execute

와 같은 Domain Model을 유지한다.

172. Language-native Design

단순 기계 번역 형태를 피한다.

예:

Python:

async for item in ...

TypeScript:

for await (...)

각 언어 관용구를 따른다.

173. SDK Release Process

Contract Change

→ SDK Generate/Implement

→ Tests

→ Release Candidate

→ Publish

→ Documentation

174. Release Artifact

각 SDK Package에는 다음이 필요하다.

- Version

- Changelog
 - License
 - Integrity
 - Documentation Link
 - Supported API Version
-

175. Package Signing

가능한 Ecosystem에서는 공식 SDK Package Publishing 계정을 보호하고 Package Integrity를 제공한다.

176. Supply Chain Security

SDK Build에는 다음을 적용할 수 있다.

- Dependency Lock
 - SBOM
 - Vulnerability Scan
 - Signed Release
 - Build Provenance
-

177. Dependency 정책

SDK는 불필요한 Dependency를 최소화한다.

특히 Core SDK에 대형 Framework를 강제로 포함하지 않는다.

178. Breaking Dependency

SDK 내부 Dependency 업데이트가 사용자 Runtime Compatibility를 깨뜨리는지 검토한다.

179. SDK Deprecation 정책

Deprecated Method에는 다음을 제공한다.

- Replacement
 - Migration Example
 - Removal Version
-

180. Security Patch

Critical Security Issue는 Patch Release로 즉시 배포할 수 있다.

필요하면 이전 SDK Version 사용을 Block/Warning할 수 있다.

181. SDK Governance

SDK Public API 변경은 다음 Review를 요구한다.

- API Owner
 - SDK Owner
 - Architecture
 - Security if relevant
 - Documentation
-

182. SDK Owner

Tier 1 SDK에는 명확한 Owner가 있어야 한다.

책임:

- Release
- Compatibility
- Security
- Documentation

- Support
 - Contract Tests
-

183. Developer Feedback

SDK에는 Usage Feedback 채널이 필요하다.

하지만 Telemetry를 강제로 사용자 코드 내용까지 수집하는 방식은 사용하지 않는다.

184. SDK Metrics

내부적으로 다음을 추적할 수 있다.

- SDK Version Distribution
- API Error by SDK
- Deprecated API Usage
- Language Distribution

Privacy/Contract를 준수한다.

185. SDK와 Marketplace

Marketplace Publisher는 SDK를 통해 다음을 수행할 수 있다.

- Package Validate
 - Package Build
 - Package Publish
 - Installation Test
-

186. Marketplace Publisher SDK

Plugin SDK 또는 별도의 Publisher Tooling으로 제공할 수 있다.

기능:

manifest validation

compatibility

signature

package

publish

187. SDK와 Agent Package

Agent Package 개발자는 Resource Schema를 SDK Type으로 생성할 수 있다.

단, SDK가 Agent Package 자체를 자동 Production Publish하지 않는다.

188. SDK와 Workflow Definition

Workflow Builder Library를 제공할 수 있다.

예:

```
const workflow = Workflow.define(...)
```

```
  .agent(...)
```

```
  .condition(...)
```

```
  .plugin(...)
```

```
  .end();
```

최종 결과는 공식 Workflow Schema여야 한다.

189. Workflow DSL

SDK DSL은 편의 기능이다.

Source of Truth는 DSL 코드가 아니라 Publish된 Workflow Resource/Schema Contract다.

또한 GUI와 DSL이 서로 다른 실행 모델을 만들어서는 안 된다.

190. Agent Builder API

SDK는 Agent Definition Builder를 제공할 수 있다.

예:

```
Agent.define({  
  goal: "...",  
  model: ...,  
  tools: ...  
});
```

최종 Serialized 결과는 Canonical Agent Schema를 따라야 한다.

191. Builder Validation

Builder가 Developer UX를 개선할 수 있지만 Server-side Validation을 대체하지 않는다.

192. SDK와 Configuration as Code

Agent/Workflow/Policy 정의를 Code Repository에서 관리할 수 있도록 SDK/CLI를 지원할 수 있다.

예:

resource file

→ validate

→ diff

→ publish

193. Diff

Resource Version 간 의미 있는 차이를 보여줄 수 있다.

예:

- Permission Added
- Tool Added
- Model Policy Changed
- Autonomy Changed

단순 JSON Text Diff보다 Semantic Diff가 유용하다.

194. Plan / Dry Run

SDK/CLI에서 Publish 전에 Dry Run을 지원할 수 있다.

예:

validate

compatibility

policy simulation

dependency check

195. SDK Acceptance Criteria

AGEX SDK는 최소 다음을 충족해야 한다.

1. 공식 API Contract를 기반으로 한다.
2. Canonical Schema를 사용한다.
3. Type-safe Resource Client를 제공한다.
4. Authentication Provider를 지원한다.
5. Tenant Context를 First-Class로 처리한다.
6. Scoped Tenant Client가 안전하다.
7. Agent Client를 제공한다.
8. Workflow Client를 제공한다.
9. Execution Handle을 제공한다.

10. Long-running Execution을 지원한다.
11. Streaming을 지원한다.
12. Knowledge Client를 제공한다.
13. Memory Client를 제공한다.
14. Plugin Client를 제공한다.
15. Model Client를 제공한다.
16. Approval/Governance Client를 제공한다.
17. Cursor Pagination을 지원한다.
18. Idempotency를 지원한다.
19. Optimistic Concurrency를 지원한다.
20. Stable Error Mapping을 제공한다.
21. Request/Correlation ID를 유지한다.
22. Webhook Signature Helper를 제공할 수 있다.
23. Plugin SDK를 분리 제공한다.
24. Model Adapter SDK를 제공한다.
25. Testing SDK/Fixtures를 제공한다.
26. TypeScript와 Python을 Tier 1로 지원한다.
27. SDK와 API 간 Contract Test가 존재한다.
28. SDK Versioning/Deprecation 정책이 존재한다.
29. Secret을 자동 Log하지 않는다.
30. Server Security를 SDK에 의존하지 않는다.

196. SDK 금지사항

다음 구조는 허용하지 않는다.

- SDK를 API Source of Truth로 사용

- TypeScript와 Python이 서로 다른 Resource 의미 사용
- Tenant ID를 Mutable Global Client State로 관리
- SDK가 Client-side Tenant Filter로 보안 구현
- Provider SDK를 Agent SDK Public API에 직접 노출
- Permission Denied 자동 Retry
- Conflict 발생 시 Silent Auto Merge
- Streaming 연결 종료 시 Execution 자동 취소
- Secret/Token Debug Log 출력
- Browser SDK에 Service Admin Credential 저장
- Generated Client 직접 수동 수정
- Plugin SDK에서 Core DB 접근 제공
- Model Adapter에서 Tool 직접 실행
- SDK가 Raw Chain-of-Thought 반환
- listAll()로 무제한 Resource 자동 Memory 로드
- 모든 Retry 요청에 새로운 Idempotency Key 생성
- SDK를 사용하면 Server Validation 생략
- SDK Middleware가 Tenant/Security Header를 임의 Override
- Community SDK를 Official Tier처럼 표시
- Deprecated 기능을 Migration 없이 갑자기 제거

197. MVP SDK 범위

AGEX 초기 Production MVP에서 우선 제공해야 할 SDK 기능은 다음과 같다.

Tier 1 Language

- TypeScript
- Python

Core Client

- Authentication
- Tenant
- Agent
- Workflow
- Execution
- Knowledge
- Plugin
- Model
- Approval

Common

- Error
- Pagination
- Idempotency
- Concurrency
- Streaming
- Request ID

Developer Tools

- Basic Testing
- Mock Model
- Mock Plugin

198. MVP 이후 확장

Stage 2 이후:

- Memory Advanced SDK

- Event/Webhook Helpers
- Workflow DSL
- Agent Builder
- Plugin SDK
- Model Adapter SDK

Stage 3:

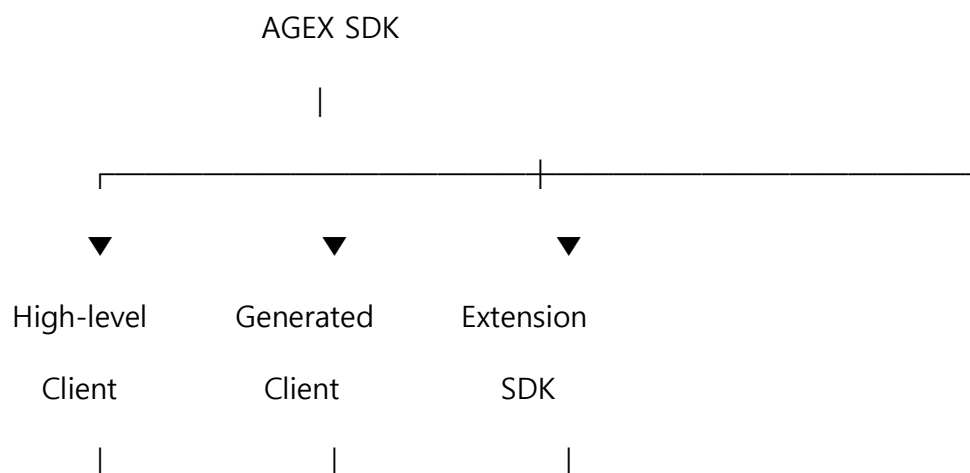
- Java
- .NET 또는 Go
- Enterprise Identity Helpers
- mTLS
- Governance SDK

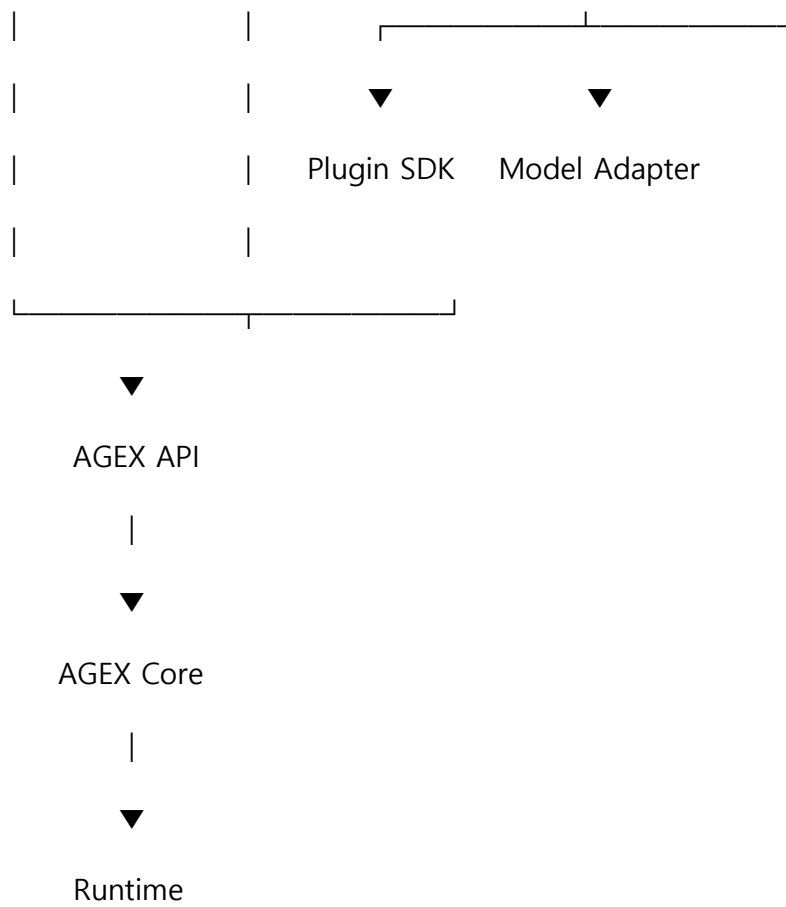
Stage 4:

- Publisher SDK
- Marketplace Package Tooling
- Additional Language SDK

199. SDK Reference Architecture

최종 구조:





200. SDK 개발 표준 흐름

API / Resource Contract



Canonical Schema



Code Generation



Generated Client



Developer-friendly Layer



Unit Test



Contract Test



Integration Test



Documentation



Release

201. SDK End-to-End 사용 시나리오

Application

→ AGEX SDK

→ Tenant Scoped Client

→ Agent Execute

→ Execution ID

→ Runtime

→ Model / Knowledge / Tool

→ Stream Events

→ Execution Completed

→ SDK Result

Application은 Provider, Runtime Worker 또는 Database를 직접 알 필요가 없다.

202. Plugin SDK 사용 시나리오

Developer

- Plugin SDK
 - Define Manifest
 - Define Action
 - Define Schemas
 - Declare Permission
 - Test Harness
 - Package
 - Validate
 - Publish
 - Marketplace/Registry
-

203. Model Adapter SDK 사용 시나리오

Provider Integration Developer

- Model Adapter SDK
 - Implement Provider Interface
 - Normalize Generate
 - Normalize Stream
 - Normalize Tool Calls
 - Normalize Usage
 - Test
 - Register Adapter
 - Model Gateway
-

204. SDK의 핵심 기술 명제

명제 1

API가 계약이고 SDK는 그 계약의 공식 언어별 표현이다.

명제 2

SDK의 목적은 HTTP 호출을 숨기는 것이 아니라 AGEX의 Resource와 Runtime 의미를 안전하게 전달하는 것이다.

명제 3

Tenant와 Security는 SDK Convenience Feature가 아니라 모든 요청에 적용되는 기본 Context다.

명제 4

Generated Client와 Developer Experience Layer를 분리해야 Contract 정확성과 사용성을 동시에 확보할 수 있다.

명제 5

Plugin SDK와 Model Adapter SDK는 AGEX Ecosystem 확장의 핵심 개발 인터페이스다.

명제 6

SDK가 편리해져도 Server Runtime과 Security Enforcement를 대체해서는 안 된다.

205. AGEX SDK의 최종 정의

AGEX SDK는 단순히 AGEX REST API를 호출하기 위한 Helper Library가 아니다.

AGEX SDK는 Agent, Workflow, Execution, Knowledge, Memory, Plugin, Model 및 Governance라는 AGEX의 공식 Resource Contract를 각 개발 언어에서 Type-safe하고 일관된 방식으로 사용할 수 있도록 제공하는 Developer Platform이다.

SDK는 Canonical Schema와 API Specification을 Source of Truth로 사용하며, 생성형 Low-level Client와 개발자 친화적인 High-level Interface를 분리하여 API Contract 정확성과 Developer Experience를 동시에 확보한다.

모든 Tenant Resource 호출에는 Tenant Context가 명시적으로 유지되어야 하며 SDK는 Authentication, Request ID, Idempotency, Pagination, Concurrency 및 Error Mapping과 같은 반복 구현을 표준화한다.

Agent와 Workflow의 Long-running Execution은 단일 HTTP Request의 수명과 분리하고,

SDK는 Execution Handle을 통해 상태 조회, Streaming, Cancel 및 완료 대기를 일관된 방식으로 제공한다.

Streaming 연결이 종료되어도 Execution이 자동 취소되지 않으며 최종 상태는 AGEX Runtime의 Execution Resource를 기준으로 한다.

AGEX SDK는 Model Provider별 API를 Application에 직접 노출하지 않고 Model Gateway의 공통 Contract를 사용하며, Plugin 개발에는 별도의 Plugin SDK, Model Provider 통합에는 Model Adapter SDK를 제공한다.

SDK는 개발자의 실수를 줄이기 위해 Client-side Validation, Typed Error, Idempotency Helper 및 Concurrency Control을 제공하지만 최종 Security와 Permission 검증은 언제나 AGEX Server와 Runtime에서 이루어진다.

TypeScript와 Python을 초기 Tier 1 SDK로 운영하고, 시장과 Enterprise 요구에 따라 Java, Go, .NET 등으로 확대한다.

SDK의 모든 공식 Release는 Canonical Schema, API Server 및 SDK 간 Contract Test를 통과해야 하며 Versioning, Compatibility, Deprecation 및 Security Patch 정책을 명확히 관리한다.

AGEX SDK는 다음 문장으로 최종 정의한다.

AGEX SDK는 AI Operating System의 모든 공개 Resource와 실행 능력을 개발자가 각 Programming Language에서 안전하고 일관되며 자동화 가능한 방식으로 사용할 수 있도록 제공하는 AGEX의 공식 Developer Platform이다.

본 문서는 AGEX Client SDK, Plugin SDK, Model Adapter SDK, Testing SDK, Versioning, Security, Streaming, Error Handling 및 Developer Experience의 공식 기준으로 사용한다.